

Kapitel 8 — Der Arbeitspunkt: von der blauen LED zum vollständigen Transistormodell

Buchentwurf zur Freigabe · Teil III: Vom Messwert zum Modell

Prof. Dr.-Ing. Ralph Wystup

Entwurf, Stand 4. August 2026

Inhaltsverzeichnis

8 Der Arbeitspunkt: von der blauen LED zum vollständigen Transistormodell	1
8.1 Sinn und Ziel dieses Kapitels	1
8.2 Das Verfahren im Eindimensionalen: die blaue LED	2
8.3 Aufstieg zum Transistor: die Fixed-Bias-Schaltung	8
8.4 Das nichtlineare System $F(V_{BE}, V_{CE}) = 0$	9
8.5 Newton-Raphson im Mehrdimensionalen	9
8.6 Die Jacobi-Matrix — vollständig hergeleitet	9
8.7 Das Programm	10
8.8 Die Bisektion stellt R_B ein	11
8.9 Ergebnis mit den gemessenen BC547-Parametern	12
8.10 Zusammenfassung	12
Übungsaufgaben zu Kapitel 8	14

8 Der Arbeitspunkt: von der blauen LED zum vollständigen Transistormodell

8.1 Sinn und Ziel dieses Kapitels

Modell (Kapitel 6) und gemessene Parameter (Kapitel 7) sind da — jetzt muss beides **eine Schaltung rechnen**. Die Aufgabe klingt bescheiden: Man bestimme den Gleichstrom-Arbeitspunkt einer Transistorstufe. Aber am erweiterten Modell ist sie **analytisch unlösbar** (Kapitel 6.11): Exponentialkennlinie, Early-Faktoren, stromabhängiges β und die implizite $V_{BE,eff}$ -Kopplung verweben die Gleichungen zu einem nichtlinearen System.

Damit die Verfahren nicht im Zweidimensionalen „vom Himmel fallen“, beginnt das Kapitel eine Etage tiefer: an der **blauen Leuchtdiode am Vorwiderstand** — ein Bauteil, eine Unbekannte, und doch das komplette Programm dieses Buches im Kleinformat: eine echte Messung, ein ideales Modell, das an ihr **scheitert**, eine physikalisch begründete **Erweiterung um den Bahnwiderstand**, die **Approximation** der Messpunkte durch Kurvenanpassung — und schließlich das **Newton-Raphson-Verfahren**, das den Arbeitspunkt der impliziten Gleichung findet. Erst dann steigen wir zum Transistor auf, wo dieselben Verfahren im $\mathbb{R}^2$ arbeiten — mit vollständiger, analytisch hergeleiteter Jacobi-Matrix, wie es SPICE intern ebenfalls tut.

Nach diesem Kapitel können Sie:

- die ideale Shockley-Diodenkennlinie hinschreiben und ihre Gültigkeitsgrenzen benennen,

- begründen, warum LEDs — insbesondere blaue GaN-LEDs — eine Erweiterung um Idealitätsfaktor und Serienwiderstand benötigen,
- Messdaten mit dem erweiterten Modell **fitten** (Approximation) und erklären, warum in der Form $V(I)$ gefittet wird,
- das **Newton-Raphson-Verfahren** auf die implizite Schaltungsgleichung anwenden — mit analytischer Ableitung, sinnvollem Startwert und numerischen Schutzmaßnahmen,
- das nichtlineare System $F(V_{BE}, V_{CE}) = 0$ der Fixed-Bias-Schaltung aufstellen und die Jacobi-Matrix vollständig herleiten,
- einen Schaltungsparameter (R_B) per Bisektion auf einen Ziel-Arbeitspunkt einstellen.

8.2 Das Verfahren im Eindimensionalen: die blaue LED

Dieser Abschnitt folgt dem Vorlesungsmanuskript „LED-Kennlinien-Approximation“ und den Begleitprogrammen `Aproximation_AP_LED_normale_Shockly.py` und `Aproximation_AP_LED_5.py`; alle Zahlenwerte sind aus der Messdatei `LED_blaue.txt` reproduziert.

8.2.1 Die Messung

Gemessen wurde die Durchlasskennlinie einer blauen LED mit dem Kennlinienschreiber (Export `LED_blaue.txt`, PN-Junction-I/V-Modus): **46 Messpunkte** (V_f, I_f) von 0 bis knapp 2,9 V bzw. 12 mA. Für die Modellanpassung werden nur Punkte mit $I_f > 0,05$ mA verwendet (35 Punkte) — unterhalb dieser Schwelle dominieren Leckströme und Messrauschen, und das Modell gilt streng nur im Durchlassbereich.

8.2.2 Die ideale Shockley-Gleichung

William Shockley leitete 1949 die Strom-Spannungs-Kennlinie des idealen p-n-Übergangs aus dem Drift-Diffusions-Modell ab:

$$I_D = I_S \left(e^{V_D / (n V_T)} - 1 \right) \quad V_T = \frac{k T}{q} \approx 25,85 \text{ mV bei } 25^\circ \text{C}$$

Für $V_D \gg V_T$ bleibt die reine Exponentialfunktion; im halblogarithmischen Diagramm eine Gerade mit **1 Dekade je $n \cdot 59,5$ mV** — dieselbe 60-mV-Regel, die in Kapitel 7 den Gummel-Plot trug. Die Gleichung gilt allerdings nur unter Idealannahmen: keine Rekombination in der Raumladungszone, keine ohmschen Bahnwiderstände, Niedriginjektion. **Alle diese Annahmen verletzt eine reale LED.**

8.2.3 Das Scheitern an der blauen LED — und seine physikalischen Gründe

Passt man die reine Shockley-Gleichung an die Messpunkte an (Programm `..._normale_Shockly.py`), erhält man:

$$I_S = 5,4 \cdot 10^{-11} \text{ A}, \quad n = 5,79.$$

Der Fit „konvergiert“ — aber auf einen **unphysikalischen Idealitätsfaktor**: Für GaN-LEDs liegt n zwischen 1,8 und 3,5; $n = 5,8$ ist keine Physik mehr, sondern der verzweifelte Versuch der Exponentialfunktion, einer Kurve zu folgen, die gar keine reine Exponentialfunktion ist. Abbildung 8.1 zeigt es: Die rote Kurve hebt zu früh ab, verfehlt das Knie und schneidet die Messpunkte nur noch.

Drei physikalische Effekte sind verantwortlich — jeder verletzt eine Idealannahme, jeder erzwingt eine Modellerweiterung:

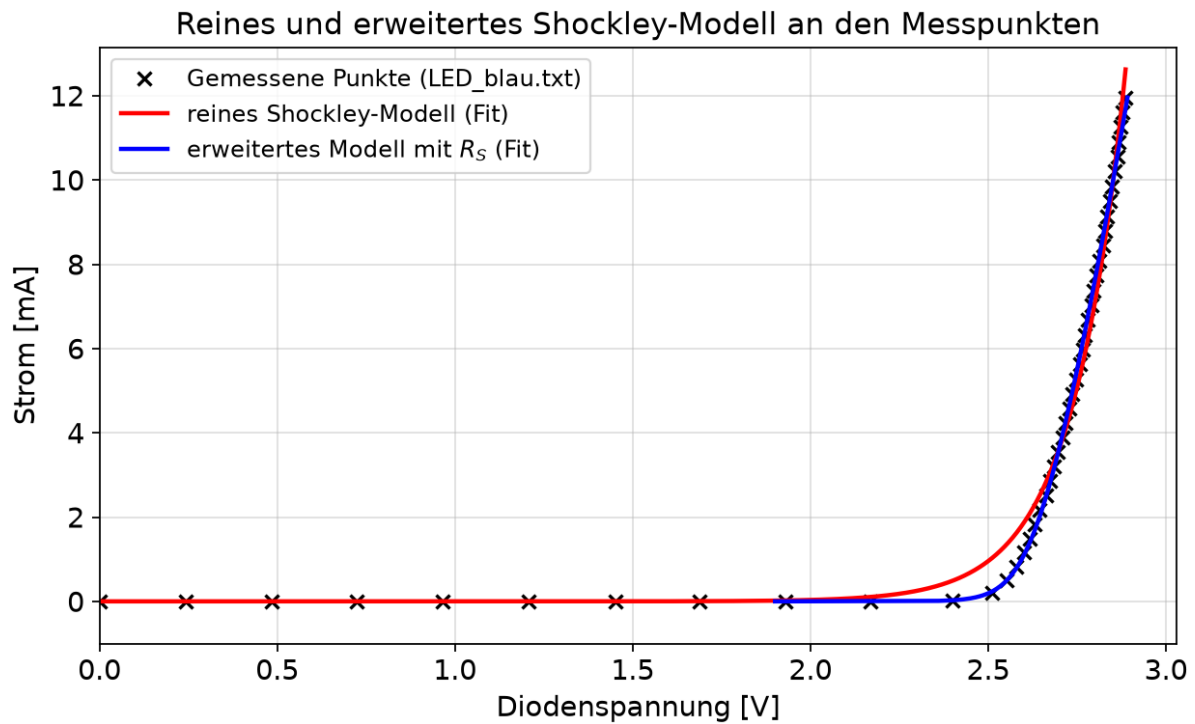


Abbildung 1: **Abbildung 8.1** — Beide Modelle an denselben Messpunkten (LED_blaue.txt): Das reine Shockley-Modell (rot) kann die gemessene Kennlinie trotz frei gefittetem n nicht wiedergeben; das um den Serienwiderstand erweiterte Modell (blau) liegt exakt auf den Messpunkten.

Beobachtung	Ursache	Modellerweiterung
$n > 1$	strahlende und SRH-Rekombination in der Raumladungszone (bei GaN zusätzlich Auger — „ABC-Rekombination“)	n als freier Parameter (1...5)
Kennlinie knickt bei hohem Strom ab, wird fast linear	ohmsche Spannungsabfälle: p-GaN-Bahngebiet (schlechte Lochbeweglichkeit!), ITO-Elektrode, Bonddrähte, Kontakte	Serienwiderstand R_S addieren
I_S extrem klein (10^{-18} A und darunter)	große Bandlücke des GaN ($E_g \approx 3,4$ eV), $I_S \sim e^{-E_g/kT}$	$\log_{10} I_S$ als Fitparameter

Die Bandlücke erklärt zugleich die hohe Flussspannung: Die Photonenenergie $E_{\text{photon}} = h c / \lambda \approx e V_F$ koppelt Farbe und Flussspannung — rote LEDs liegen bei 1,6-2,0 V, blaue InGaN/GaN-LEDs bei 2,8-3,5 V. **Blaue LEDs haben damit die höchste Flussspannung aller sichtbaren LEDs — und den stärksten R_S -Einfluss**, denn dessen Abfall $I_D R_S$ wächst linear mit dem Strom.

8.2.4 Die erweiterte Shockley-Gleichung — in der Form $V(I)$

Das erweiterte Modell ist die ideale Shockley-Diode **in Reihe mit dem parasitären Serienwiderstand R_S** . Aufgeschrieben wird es bewusst als Spannung in Abhängigkeit vom Strom:

$$V_D = n V_T \ln \left(\frac{I_D}{I_S} + 1 \right) + I_D R_S$$

Warum $V(I)$ statt $I(V)$? Der Strom variiert bei einer LED über etwa fünf Größenordnungen, die Spannung nur zwischen ca. 2,5 und 3,5 V. Ein Fehler von 1 % in V_D entspricht wegen der Exponentialfunktion bis zu einem Faktor 10 in I_D . Die Methode der kleinsten Quadrate minimiert den Fehler der **abhängigen** Variablen — wählt man V als abhängige Größe, sind die Residuen gleichmäßig verteilt und der Fit gut konditioniert. Aus demselben Grund wird nicht I_S direkt gefittet, sondern $\log_{10} I_S$: Werte um 10^{-20} A wären als direkte Parameter numerisch miserabel konditioniert, ihr Logarithmus (−20 ...) ist es nicht.

8.2.5 Die Approximation: Fit an die Messpunkte

`scipy.optimize.curve_fit` bestimmt die drei Parameter $\theta = (\log_{10} I_S, n, R_S)$ durch Minimieren der Residuenquadratsumme $\sum_i [V_{\text{gemessen},i} - V_{\text{Modell}}(I_i, \theta)]^2$, mit den Startwerten und physikalisch begründeten Grenzen des Originalprogramms ($\log_{10} I_S \in [-35, -10]$, $n \in [1, 5]$, $R_S \in [0, 200] \Omega$). Ergebnis für die vermessene blaue LED (Unsicherheiten aus der Kovarianzmatrix):

Parameter	Wert	Einordnung (typisch für blaue GaN-LEDs)
$\log_{10}(I_S/\text{A})$	$-25,83 \pm 0,72 \rightarrow$ $I_S \approx 1,5 \cdot 10^{-26} \text{ A}$	noch unter dem Typbereich $10^{-20} \dots 10^{-16} \text{ A}$ — I_S ist eine extreme Extrapolationsgröße und stark mit n korreliert ($\pm 0,72$ Dekaden!)
n	$1,898 \pm 0,060$	im Typbereich 1,8...3,5 □
R_S	$16,0 \pm 0,4 \Omega$	im Typbereich 5...50 Ω □

Die blaue Kurve in Abbildung 8.1 ist dieses Modell: Sie liegt über den gesamten Messbereich **auf** den Punkten. Der Kontrast zum reinen Modell ist die Kernbotschaft des Abschnitts: **Erst der Bahnwiderstand macht die LED approximierbar** — mit Parametern, die dann auch physikalisch deutbar sind.

8.2.6 Newton-Raphson: der Arbeitspunkt der impliziten Gleichung

Nun die Schaltung: Spannungsquelle U_{ges} , Vorwiderstand R , LED. Die Maschenregel mit eingesetztem Modell lautet

$$U_{ges} = I_D R + n V_T \ln\left(\frac{I_D}{I_S} + 1\right) + I_D R_S.$$

Diese Gleichung ist **implizit in** I_D — der Strom steht linear *und* im Logarithmus; eine geschlossene Auflösung existiert nur über die unpraktische Lambert-W-Funktion. Also numerisch: Als Fehlerfunktion (Residuum) dient die Abweichung von der Maschenregel,

$$f(I_D) = n V_T \ln\left(\frac{I_D}{I_S} + 1\right) + I_D (R + R_S) - U_{ges} \stackrel{!}{=} 0,$$

mit der **analytisch** berechneten Ableitung

$$f'(I_D) = \frac{n V_T}{I_D + I_S} + R + R_S$$

(analytisch statt numerisch: exakt, schnell, keine Schrittweitenprobleme). Die Newton-Raphson-Iteration — Tangente an f , Schnitt mit der Nullachse, quadratische Konvergenz —

$$I_D^{(k+1)} = I_D^{(k)} - \frac{f(I_D^{(k)})}{f'(I_D^{(k)})}$$

braucht noch einen Startwert. Das Originalprogramm verwendet den **gedämpften Kurzschlussstrom** $I_D^{(0)} = U_{ges}/(R + R_S + 1)$ — als läge statt der LED ein 1- Ω -Widerstand in der Masche; das liegt für typische LED-Schaltungen nahe genug an der Lösung. Der Kern in Python (wortgetreu aus Aproximation_AP_LED_5.py):

```
def solve_for_Id(Uges, R, log10_Is, n, Rs, Vt):
    Is = 10**log10_Is
    Id = Uges / (R + Rs + 1.0)      # Startwert: gedaempfter Kurzschlussstrom
    for _ in range(1000):
```

```

if Id <= 0:
    Id = 1e-12 # Schutz: ln fuer Id <= 0 nicht definiert
f = n*Vt*np.log(Id/Is + 1) + Id*Rs + Id*R - Uges
df = (n*Vt)/(Id + Is) + Rs + R
Id_new = Id - f / df
if abs(Id_new - Id) < 1e-7: # Toleranz 0,1 uA
    return Id_new if Id_new > 0 else None
Id = Id_new
return None # keine Konvergenz -> Warnung im Programm

```

Numerische Sicherheitsmaßnahmen (aus dem Originalprogramm — so arbeitet robuste Numerik): $I_D \leq 0$ wird auf 10^{-12} A zurückgesetzt (Logarithmus-Schutz); $|f'| \approx 0$ wird abgefangen; nach 1000 Iterationen ohne Konvergenz gibt es eine **Warnung statt eines stillen Fehlers**; ein Ergebnis $I_D \leq 0$ wird als physikalisch ungültig verworfen; und $V_D \geq U_{ges}$ löst eine Plausibilitätswarnung aus.

Der echte Lauf mit den gefitteten Parametern, $U_{ges} = 5$ V, $R = 100$ Ω :

k	$I_D^{(k)}$
0 (Start)	42,719 mA
1	19,512 mA
2	19,611473 mA
3	19,611478 mA — konvergiert

Drei Iterationen; von Schritt 1 auf 2 gewinnt das Verfahren vier gültige Stellen, von 2 auf 3 die restlichen — die quadratische Konvergenz in Aktion.

8.2.7 Arbeitspunkt und Lastgerade

Grafisch ist die Lösung der Schnittpunkt der LED-Kennlinie mit der **Lastgeraden** $V_D = U_{ges} - I_D R$ (Achsenschnittpunkte: Leerlauf U_{ges} auf der Spannungs-, Kurzschluss U_{ges}/R auf der Stromachse):

Größe	Wert
V_D	3,039 V
I_D	19,61 mA
$U_R = I_D R$	1,96 V
$P_R = I_D^2 R$	38,5 mW

Dimensionierungsregel (aus dem Vorlesungsmanuskript): Für die schnelle Vorwiderstands-Wahl genügt $R = (U_{ges} - V_F)/I_{D,soll}$. Beispiel: $U_{ges} = 5$ V, $V_F = 3,2$ V (blaue LED bei 20 mA), $I_{D,soll} = 20$ mA $\rightarrow R = 90$ $\Omega \rightarrow$ Normwert 100 Ω . Die exakte Rechnung oben bestätigt die Auslegung: Mit dem Normwert fließen 19,6 mA.

8.2.8 Was der Transistor davon erbt

Der Aufstieg ins Zweidimensionale ändert die Struktur nicht:

- **Residuum $\rightarrow$ Ableitung $\rightarrow$ Iteration** bleibt die Dreiteilung; aus f und f' werden der Vektor F und die Jacobi-Matrix J , aus der Division f/f' das lineare Gleichungssystem $J \Delta x = -F$.
- Der **Serienwiderstand** kehrt wieder: Was bei der LED R_S ist, ist beim Transistor der innere Basiswiderstand $R_{B,int}$ — beide machen die Gleichungen implizit.

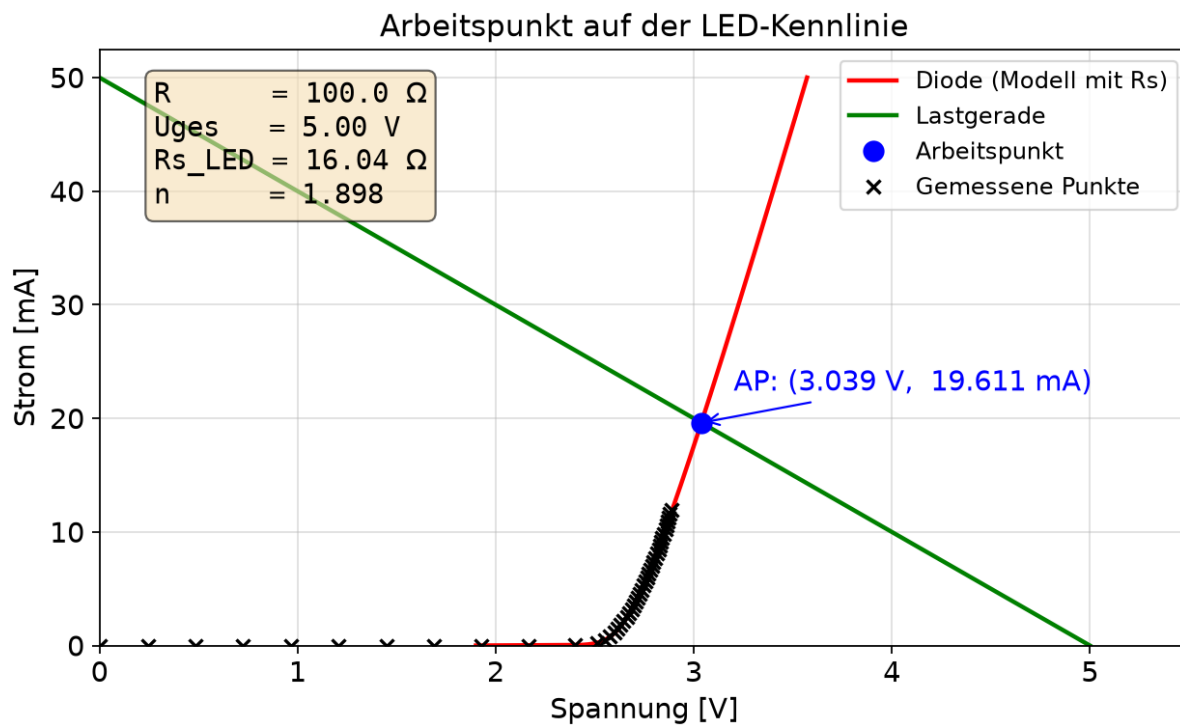


Abbildung 2: **Abbildung 8.2** — Arbeitspunkt auf der LED-Kennlinie (Reproduktion des Originaldiagramms): Modellkurve, Lastgerade, Arbeitspunkt — und die Messpunkte zum Abgleich. Der Arbeitspunkt liegt mitten im vermessenen Bereich, das Modell interpoliert hier, es extrapoliert nicht.

- Die **Formulierungswahl** ist jeweils bewusst: Bei der LED löst $V(I)$ das Fit-Konditionierungsproblem; beim Transistor arbeiten wir in den Klemmenspannungen (V_{BE} , V_{CE}), weil Maschenregeln und Modell dort am natürlichsten zusammentreffen.

8.3 Aufstieg zum Transistor: die Fixed-Bias-Schaltung

Als Beispielschaltung dient die einfachste Arbeitspunkteinstellung — die **Fixed-Bias-Schaltung** (Basisvorwiderstand, Abbildung 8.3): Versorgung $V_{CC} = 25\text{ V}$, Kollektorwiderstand $R_C = 100\ \Omega$, Basisspannung $V_{BB} = V_{CC}$, Basisvorwiderstand R_B (gesucht).

Fixed-Bias-Schaltung: R_B stellt den Arbeitspunkt ein

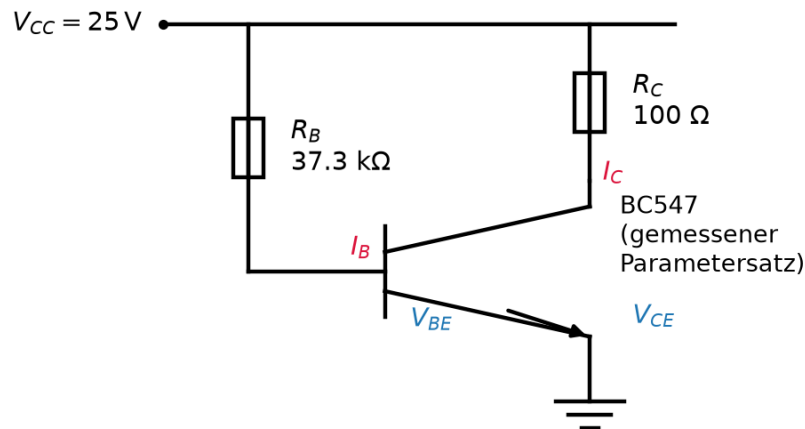


Abbildung 3: **Abbildung 8.3** — Die Fixed-Bias-Schaltung. R_B prägt den Basisstrom ein und stellt damit den Arbeitspunkt ein; R_C setzt den Kollektorstrom in die Ausgangsspannung um.

Zwei unabhängige Maschen liefern zwei Gleichungen — und damit genau die Zahl, die wir für die zwei Unbekannten V_{BE} und V_{CE} brauchen:

Basiskreis:

$$V_{BB} = I_B R_B + V_{BE} \Rightarrow I_B = \frac{V_{BB} - V_{BE}}{R_B}$$

Kollektorkreis (Lastgerade):

$$V_{CC} = I_C R_C + V_{CE} \Rightarrow I_C = \frac{V_{CC} - V_{CE}}{R_C}$$

Die Lastgerade verbindet die Punkte $(0, V_{CC}/R_C) = (0, 250\text{ mA})$ und $(V_{CC}, 0) = (25\text{ V}, 0)$. Der Arbeitspunkt ist ihr Schnittpunkt mit der Transistorkennlinie — wie bei der LED, nur dass die „Kennlinie“ jetzt das volle Modell aus Kapitel 6.8 ist.

8.4 Das nichtlineare System $F(V_{BE}, V_{CE}) = 0$

Gesucht ist das Paar (V_{BE}, V_{CE}) , das **gleichzeitig** die Schaltungsgleichungen und das Transistormodell erfüllt. Wie bei der LED schreibt man Residuen — jetzt zwei:

Gleichung 1 — Basiskreis trifft Basisstrom-Modell:

$$f_1(V_{BE}, V_{CE}) = I_B - \frac{I_S}{\beta_{\text{eff}}} e^{V_{BE, \text{eff}}/(nV_T)} \left(1 + \frac{V_{CE}}{V_{AB}}\right) = 0$$

Gleichung 2 — Lastgerade trifft Kollektorstrom-Modell:

$$f_2(V_{BE}, V_{CE}) = I_C - I_S e^{V_{BE, \text{eff}}/(nV_T)} \left(1 + \frac{V_{CE}}{V_A}\right) = 0$$

mit den inneren Größen (alle aus den Kapiteln 6 und 8.3):

$$I_B = \frac{V_{BB} - V_{BE}}{R_B}, \quad I_C = \frac{V_{CC} - V_{CE}}{R_C}, \quad V_{BE, \text{eff}} = V_{BE} - I_B R_{B, \text{int}}, \quad \beta_{\text{eff}} = \frac{\beta_F}{\sqrt{1 + I_C/I_{KF}}}.$$

Warum keine Gleichung allein lösbar ist: f_1 enthält über β_{eff} den Kollektorstrom (also V_{CE}), f_2 über $V_{BE, \text{eff}}$ den Basisstrom (also V_{BE}) — die Gleichungen sind **wechselseitig verkoppelt**. Ein geschlossener Ausdruck existiert nicht; das ist keine mathematische Schwäche, sondern die direkte Folge der in Kapitel 6 eingebauten Physik.

8.5 Newton-Raphson im Mehrdimensionalen

Das Verfahren aus 8.2.6 überträgt sich wörtlich; nur wird aus der Ableitung die **Jacobi-Matrix**:

$$x_{k+1} = x_k - J(x_k)^{-1} F(x_k), \quad x_k = \begin{bmatrix} V_{BE} \\ V_{CE} \end{bmatrix}, \quad F = \begin{bmatrix} f_1 \\ f_2 \end{bmatrix}, \quad J = \frac{\partial F}{\partial x}.$$

Praktisch invertiert man J **nie**, sondern löst das lineare System

$$J(x_k) \Delta x = -F(x_k), \quad x_{k+1} = x_k + \Delta x$$

(im Programm `numpy.linalg.solve`; Abbruch bei $\|F(x_k)\| < 10^{-10}$). Für den BJT-Arbeitspunkt ist $[V_{BE}, V_{CE}]_0 = [0,65 \text{ V}, V_{CC}/2]$ praktisch immer ausreichend: Die Diodenspannung eines leitenden Silizium-Übergangs ist nie weit von 0,65 V entfernt — der Startwert steckt, wie bei der LED der gedämpfte Kurzschlussstrom, schon im physikalischen Vorwissen.

8.6 Die Jacobi-Matrix — vollständig hergeleitet

Die vier Elemente sind Schulableitungen — wenn man eine innere Ableitung nicht vergisst. Mit $I_B = (V_{BB} - V_{BE})/R_B$ gilt

$$V_{BE, \text{eff}} = V_{BE} - \frac{V_{BB} - V_{BE}}{R_B} R_{B, \text{int}} \Rightarrow \boxed{\frac{dV_{BE, \text{eff}}}{dV_{BE}} = 1 + \frac{R_{B, \text{int}}}{R_B}}$$

— größer als 1, weil ein steigendes V_{BE} zugleich I_B senkt und damit den Abzugsterm verkleinert: beide Effekte ziehen $V_{BE, \text{eff}}$ nach oben. Damit die vier Elemente (Abkürzung $E \equiv e^{V_{BE, \text{eff}}/(nV_T)}$, $b \equiv \beta_{\text{eff}}$):

$$J_{11} = \frac{\partial f_1}{\partial V_{BE}} = -\frac{1}{R_B} - \frac{I_S}{b} E \frac{dV_{BE,eff}}{dV_{BE}} \frac{1}{nV_T} \left(1 + \frac{V_{CE}}{V_{AB}}\right)$$

$$J_{12} = \frac{\partial f_1}{\partial V_{CE}} = -\frac{I_S}{b} E \frac{1}{V_{AB}} \quad (\text{nur der Early-Faktor hängt von } V_{CE})$$

$$J_{21} = \frac{\partial f_2}{\partial V_{BE}} = -I_S E \frac{dV_{BE,eff}}{dV_{BE}} \frac{1}{nV_T} \left(1 + \frac{V_{CE}}{V_A}\right)$$

$$J_{22} = \frac{\partial f_2}{\partial V_{CE}} = -\frac{1}{R_C} - I_S E \frac{1}{V_A} \quad (\text{Lastgerade: } dI_C/dV_{CE} = -1/R_C)$$

(Die schwache V_{CE} -Abhängigkeit von b über I_C wird hier, wie in der SPICE-Praxis üblich, in der Jacobi-Matrix vernachlässigt — sie beschleunigt nur die Konvergenz, ändert aber die Lösung nicht, denn **die Lösung wird allein von $F = 0$ bestimmt, nicht von J** . Eine ungenaue Jacobi-Matrix kostet Iterationen, nie Richtigkeit.)

8.7 Das Programm

Der Kern in unmittelbarer Übersetzung der Formeln — Modellfunktionen aus Kapitel 6.9 vorausgesetzt:

```
def F(x, RB):
    """Residuenvektor [f1, f2] des nichtlinearen Systems."""
    vbe, vce = x
    ib = (VBB - vbe) / RB          # Basiskreis-Masche
    ic = (VCC - vce) / RC          # Lastgerade
    veff = vbe - ib * RBI          # innerer Basiswiderstand
    ex = np.exp(veff / (NF * VT))
    b = beta_eff(ic)               # Webster-Formel
    f1 = ib - (IS / b) * ex * (1 + vce / VAR)
    f2 = ic - IS * ex * (1 + vce / VAF)
    return np.array([f1, f2])
```

```
def J(x, RB):
    """Jacobi-Matrix, analytisch (Abschnitt 8.6)."""
    vbe, vce = x
    ib = (VBB - vbe) / RB
    ic = (VCC - vce) / RC
    ex = np.exp((vbe - ib * RBI) / (NF * VT))
    b = beta_eff(ic)
    dveff = 1 + RBI / RB          # innere Ableitung!
    j11 = -1/RB - (IS/b) * ex * dveff / (NF*VT) * (1 + vce/VAR)
    j12 = -(IS/b) * ex / VAR
    j21 = -IS * ex * dveff / (NF*VT) * (1 + vce/VAF)
    j22 = -1/RC - IS * ex / VAF
    return np.array([[j11, j12], [j21, j22]])
```

```
def newton(RB, x0=(0.65, VCC/2), tol=1e-10):
    x = np.array(x0, float)
    for k in range(100):
        r = F(x, RB)
        if np.linalg.norm(r) < tol:
```

```

    return x, k
    x = x + np.linalg.solve(J(x, RB), -r)
return x, 99

```

Rechenbeispiel 8.1 — der Startpunkt, von Hand nachgeprüft. Am Startwert $[0,65\text{ V}, 12,5\text{ V}]$ mit $R_B = 37,3\text{ k}\Omega$ (gemessene BC547-Parameter): $I_B = (25 - 0,65)/37,3\text{ k} = 652,9\text{ }\mu\text{A}$, $I_C = 125\text{ mA}$, $V_{BE,\text{eff}} = 650 - 9,8 = 640,2\text{ mV}$, $\beta_{\text{eff}} = 290/\sqrt{1 + 0,125/0,08} = 181$. Damit $f_1 = 6,4 \cdot 10^{-4}$ und $f_2 = 0,122$ — beide Residuen weit von Null: Der Startpunkt liegt daneben, wie erwartet. Nach **48 Newton-Raphson-Schritten** ist $\|F\| < 10^{-10}$ (Abbildung 8.5 links) — jede Iteration in Millisekunden.

8.8 Die Bisektion stellt R_B ein

Bisher war R_B gegeben. Die Entwurfsaufgabe lautet umgekehrt: **Finde R_B so, dass $V_{CE} = V_{CC}/2$** — die Mitte der Lastgeraden, maximaler symmetrischer Aussteuerbereich für den späteren Verstärker (Kapitel 12).

Dafür verwendet das Vorlesungsprogramm (Erweitere_Spice_Parameter_1.py) das **Bisektionsverfahren** (Intervallschachtelung). Es setzt nur zweierlei voraus: Stetigkeit und einen **Vorzeichenwechsel** der Zielabweichung $g(R_B) = V_{CE}(R_B) - V_{CC}/2$ über dem Suchintervall. Den garantiert hier die Physik als **Monotoniekette**:

$$R_B \uparrow \Rightarrow I_B \downarrow \Rightarrow I_C \downarrow \Rightarrow V_{RC} = I_C R_C \downarrow \Rightarrow V_{CE} = V_{CC} - V_{RC} \uparrow$$

Algorithmus: Intervallmitte R_B^* probieren $\rightarrow$ innen löst Newton-Raphson die Schaltung $\rightarrow$ je nach Vorzeichen von $V_{CE} - V_{CC}/2$ die untere oder obere Intervallgrenze nachziehen $\rightarrow$ wiederholen. Nach k Schritten ist R_B auf $(b - a)/2^k$ eingegrenzt — **ein Bit Genauigkeit je Schritt, garantiert**, aber eben nur linear konvergent. So sieht der tatsächliche Suchlauf aus (gemessene BC547-Parameter, Startintervall 10...500 k Ω):

Schritt	R_B -Versuch	V_{CE} (Newton-Raphson)	Entscheidung
1	255,0 k Ω	22,35 V	zu groß $\rightarrow$ absenken
2	132,5 k Ω	20,36 V	zu groß $\rightarrow$ absenken
3	71,2 k Ω	17,35 V	zu groß $\rightarrow$ absenken
4	40,6 k Ω	13,26 V	zu groß $\rightarrow$ absenken
5	25,3 k Ω	8,47 V	zu klein $\rightarrow$ anheben
6	33,0 k Ω	11,32 V	zu klein $\rightarrow$ anheben
7	36,8 k Ω	12,37 V	zu klein $\rightarrow$ anheben
8	38,7 k Ω	12,83 V	zu groß $\rightarrow$ absenken
9	37,8 k Ω	12,61 V	zu groß $\rightarrow$ absenken
10	37,3 kΩ	12,49 V	Ziel erreicht

Das Intervall halbiert sich stur — nach zehn Schritten von 490 k Ω auf unter 1 k Ω ($490/2^{10} \approx 0,5$). Hier verschachteln sich die beiden Verfahren des Kapitels: **außen die garantierte, aber langsame Bisektion über den Entwurfparameter; innen der schnelle Newton-Raphson über die Schaltungsgrößen**. Diese Arbeitsteilung „außen robust, innen schnell“ kehrt in Optimierung und Simulation ständig wieder.

Programmstruktur im Überblick (vollständiges Programm kap08_rechnung.py im Begleitmaterial; Original der Vorlesung: Erweitere_Spice_Parameter_1.py):

Programmteil	Funktion
Parameterblock	gemessene BC547-Parameter (Kap. 7.8) + Schaltungswerte

Programmteil	Funktion
beta_eff(ic)	Webster-Formel (Kap. 6.6)
F(x, RB)	Residuenvektor $[f_1, f_2]$ (Abschn. 8.4)
J(x, RB)	analytische Jacobi-Matrix (Abschn. 8.6)
newton(RB, x0)	Newton-Raphson mit linalg.solve (Abschn. 8.5/8.7)
Bisektionsschleife	R_B -Suche auf $V_{CE} = V_{CC}/2$ (Abschn. 8.8)
Ausgabe/Plots	Arbeitspunkttable, Lastgerade, Konvergenzkurven

8.9 Ergebnis mit den gemessenen BC547-Parametern

Mit dem Parametersatz aus Kapitel 7.8 ($I_S = 5,0 \cdot 10^{-14} \text{ A}$, $n = 1,01$, $\beta_F = 290$, $V_A = 95 \text{ V}$, $V_{AB} = 190 \text{ V}$, $I_{KF} = 80 \text{ mA}$, $R_{B,int} = 15 \Omega$) liefert die Bisektion:

Größe	Wert	Bemerkung
R_B	37,3 kΩ	Ergebnis der Bisektion
V_{BE}	752,2 mV	Klemmenspannung
$V_{BE,eff}$	742,4 mV	9,8 mV Abfall an $R_{B,int}$
V_{CE}	12,49 V	Ziel $V_{CC}/2$ getroffen
I_B	650,5 μA	
I_C	125,1 mA	Mitte der Lastgeraden
β_{eff}	181	statt $\beta_F = 290$!
I_C/I_{KF}	1,56	tief in der Hochinjektion

Vergleich mit der idealen Handabschätzung. Ohne Early, $R_{B,int}$ und Webster rechnet man in einer Minute: $I_C^{Ziel} = V_{CC}/(2R_C) = 125 \text{ mA}$, $V_{BE} \approx nV_T \ln(I_C/I_S) = 746 \text{ mV}$, $I_B = I_C/\beta_F = 431 \mu\text{A}$,

$$R_B^{ideal} = \frac{V_{BB} - V_{BE}}{I_B} = \frac{25 - 0,746}{431 \mu\text{A}} \approx 56,3 \text{ k}\Omega.$$

Das vollständige Modell sagt aber **37,3 kΩ** — die Abschätzung liegt um **34 % daneben**. Der Grund steht in der Ergebnistabelle: Bei 125 mA ist $I_C/I_{KF} = 1,56$ — der BC547 arbeitet weit oberhalb seines gemessenen Kniestroms, β bricht von 290 auf 181 ein, und der Basisstrom muss um eben diesen Faktor größer sein als ideal gedacht. **Ob die Faustformel trägt, entscheidet also nicht die Formel, sondern der in Kapitel 7 gemessene Wert von I_{KF}** — mit dem Demo-Wert 0,5 A wäre der Fehler unter 10 % geblieben. Messung, Modell und Numerik: erst die ganze Kette macht die Aussage richtig.

(Praktische Randnotiz: $I_C = 125 \text{ mA}$ überschreitet die Dauergrenze $I_{C,max} = 100 \text{ mA}$ des BC547 und liegt oberhalb des in Kapitel 7 vermessenen Strombereichs — das Modell wird hier bewusst extrapoliert, um den I_{KF} -Effekt drastisch zu zeigen. Ein realer Entwurf würde R_C größer wählen; Aufgabe 5 rechnet genau diesen Fall.)

8.10 Zusammenfassung

1. **Die blaue LED zeigt das ganze Programm im Kleinen:** Die ideale Shockley-Gleichung scheitert an der realen Kennlinie (Fit flieht in ein unphysikalisches $n = 5,8$); erst die Erweiterung um den Serienwiderstand R_S macht die Messpunkte approximierbar — mit physikalisch deutbaren Parametern ($n = 1,90$, $R_S = 16 \Omega$).
2. Der Fit erfolgt in der Form $V(I)$ (Konditionierung!), der Arbeitspunkt folgt aus der **impliziten** Maschengleichung per **Newton-Raphson** — analytische Ableitung, physikalisch motivierter Startwert, Schutzmaßnahmen; Konvergenz in drei Iterationen.

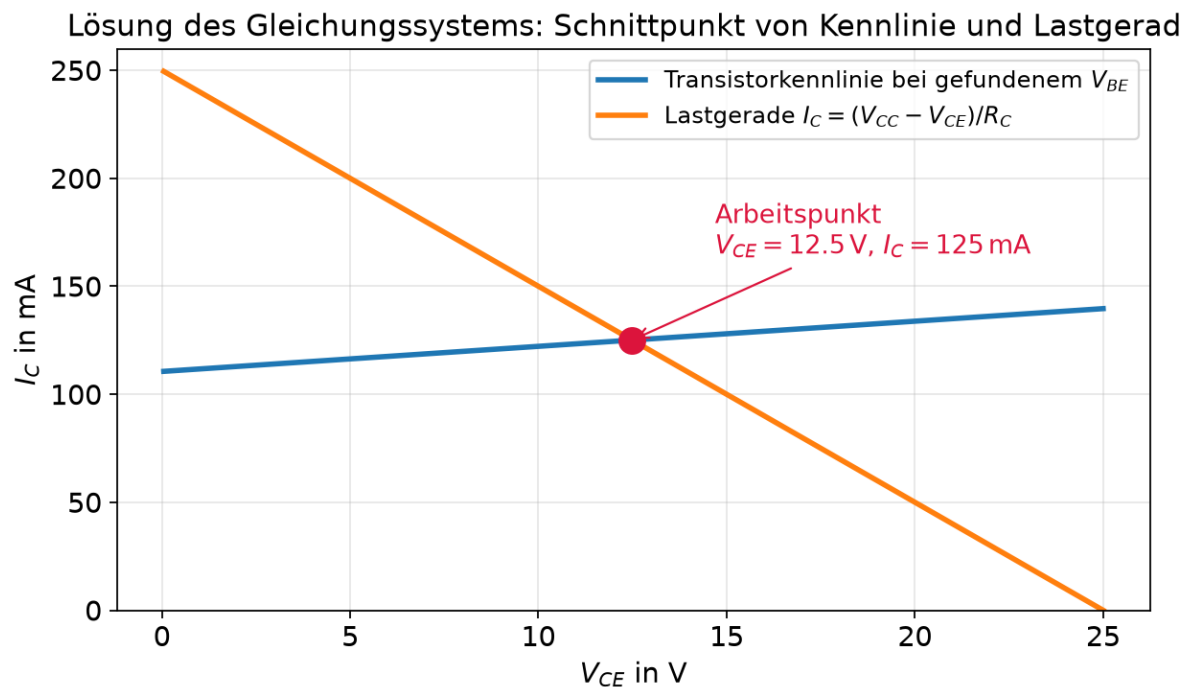


Abbildung 4: **Abbildung 8.4** — Die Lösung als Bild: Transistorkennlinie (beim gefundenen V_{BE}) und Lastgerade schneiden sich im Arbeitspunkt $V_{CE} = 12,5 \text{ V}$, $I_C = 125 \text{ mA}$. Ehrlichkeitshinweis: Die Kennlinie ist das in Kapitel 7 **an die Messung angepasste Modell**; der Arbeitspunkt selbst liegt oberhalb des vermessenen Strombereichs — das Modell wird hier also extrapoliert (deshalb sind hier keine Messpunkte einzeichnenbar; bei der LED in Abbildung 8.2 lag der Arbeitspunkt dagegen mitten im Messbereich).

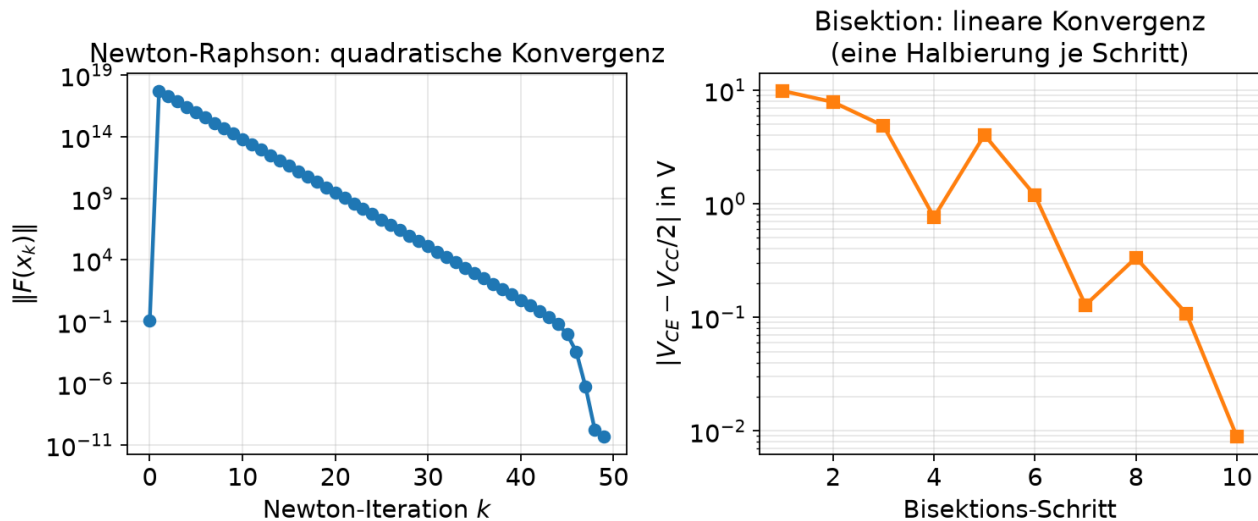


Abbildung 5: **Abbildung 8.5** — Konvergenzverhalten: links Newton-Raphson. Vom Startwert $V_{BE} = 0,65$ V aus schießt der erste Schritt über die e -Funktion hinaus; danach folgen rund 42 Schritte gedämpfter Abstieg mit dem Faktor $1/e$ je Schritt (eine Gerade im logarithmischen Bild, denn jeder Schritt senkt V_{BE} um genau nV_T), und erst die letzten vier Schritte sind quadratisch: bei $k = 48$ fällt $\|F\|$ unter 10^{-10} . Rechts die Bisektion (linear — eine Halbierung je Schritt), die nach zehn Schritten im Toleranzband liegt. Die Überschrift im Bild selbst nennt nur den letzten, quadratischen Abschnitt; sie stammt aus kap08_rechnung.py und ist dort unverändert geblieben.

3. Beim Transistor wird daraus das System $F(V_{BE}, V_{CE}) = 0$ mit analytischer Jacobi-Matrix (innere Ableitung $1 + R_{B,int}/R_B$!); außen stellt die **Bisektion** den Entwurfsparameter R_B ein — „außen robust, innen schnell“.
4. Mit den **gemessenen** Parametern verfehlt die ideale Handabschätzung das Ergebnis um 34 % — die Hochinjektion ($I_C \approx 1,6 I_{KF}$) ist im Arbeitspunkt real wirksam. Faustformeln bleiben wertvoll als Startwert und Plausibilitätstest, nicht als Ergebnis.

Kapitel 9 schließt den Kreis: Dieselben gemessenen Parameter wandern als .model-Karte in LTspice und als Blockschaltbild nach Simulink — und beide Simulatoren müssen denselben Arbeitspunkt liefern wie unser eigener Newton-Raphson-Löser.

Übungsaufgaben zu Kapitel 8

(Aufgaben 1-3 aus dem LED-Vorlesungsmanuskript)

1. **Idealitätsfaktor:** Eine blaue LED zeigt im halblogarithmischen I-V-Diagramm eine Steigung von 1 Dekade pro 120 mV. Man berechne den Idealitätsfaktor n bei $T = 25^\circ\text{C}$. (Hinweis: $\Delta V = n V_T \ln 10 = n \cdot 59,5 \text{ mV}$.)
2. **Vorwiderstand dimensionieren:** $U_{ges} = 3,3$ V, blaue LED mit $V_F = 3,05$ V bei $I_F = 10$ mA. Man berechne den benötigten Vorwiderstand, die an ihm entstehende Verlustleistung und den passenden E12-Normwert.
3. **Newton-Raphson per Hand:** Mit $n = 2$, $I_S = 10^{-20}$ A, $R_S = 20 \Omega$, $R = 100 \Omega$, $U_{ges} = 5$ V, $V_T = 25,85$ mV führe man drei Iterationen per Taschenrechner durch (Startwert $I_D^{(0)} = U_{ges}/(R + R_S + 1) = 41,3$ mA).
4. **Jacobi-Element:** Man leite J_{12} selbst her und begründe, warum dort — anders als in J_{11} — keine innere Ableitung auftritt.
5. **Realistischer Entwurf:** Man wiederhole die Bisektion mit $R_C = 1 \text{ k}\Omega$ (also $I_C^{\text{Ziel}} = 12,5$ mA).

Wie groß ist jetzt I_C/I_{KF} , wie nahe rückt β_{eff} an β_F , und auf wie viel Prozent verbessert sich die ideale R_B -Abschätzung?

6. **Serienstreuung (aus Kapitel 7 übernommen):** β_F variere über die BC547-Klassengrenzen 110...800. Man berechne R_B je Fall und erkläre, warum die Fixed-Bias-Schaltung für die Serienfertigung ungeeignet ist — und welche Schaltungsmaßnahme hilft (Vorgriff auf Kapitel 12: Emitterwiderstand).